

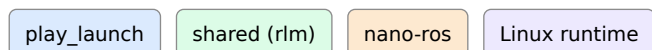
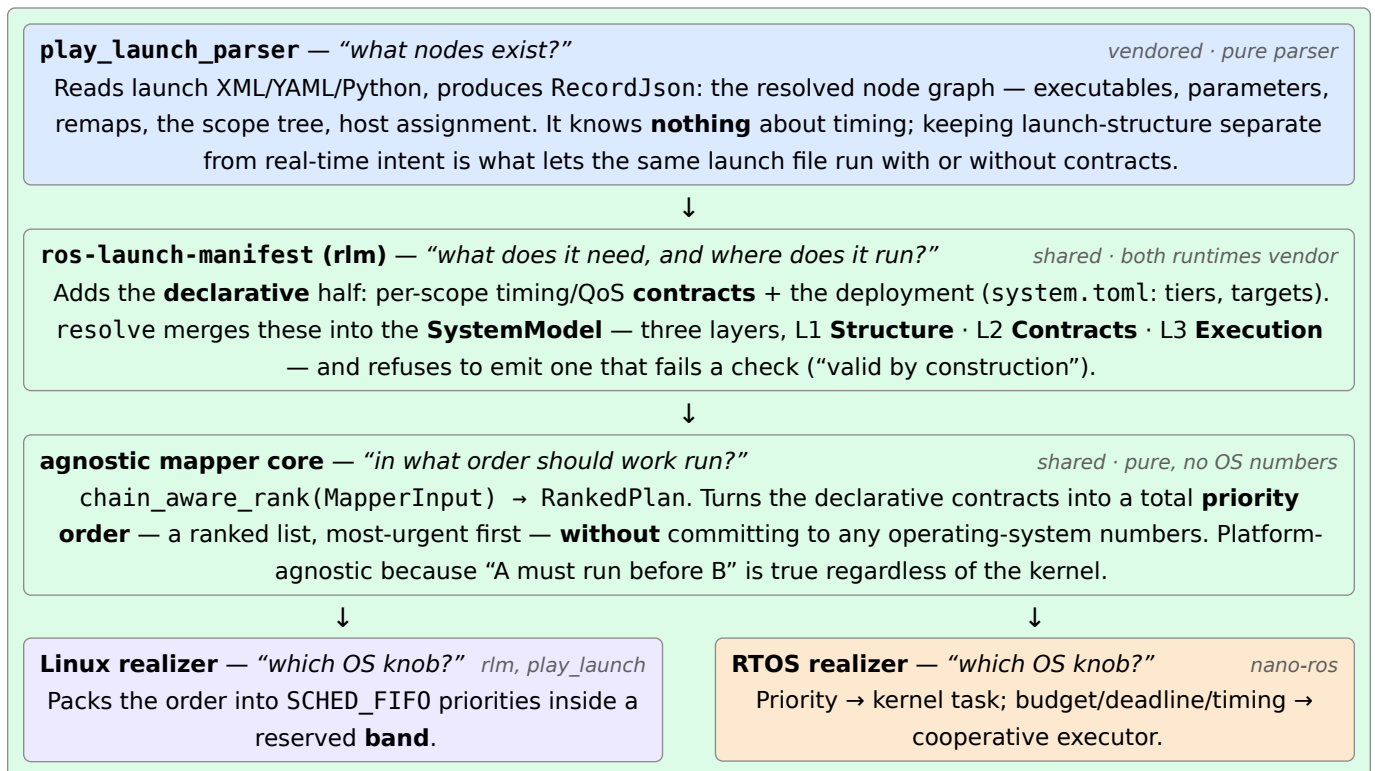
Launch/Contract → Linux/RTOS Scheduler Mapping

nano-ros internal technical report · 2026-07-22

A ROS 2 system is a graph of nodes exchanging messages. Making it **real-time** means deciding, for every processor, which node's work runs first when several are ready — and detecting when a deadline is about to be missed. This pipeline derives those decisions automatically from two declarations: the ROS **launch** description (which nodes exist and how they are wired) and a set of **timing contracts** (how fast each path must run, how stale its data may be, how important it is). It then emits concrete scheduler configuration for two very different targets — a Linux host with the full sched(7) machinery, and a bare-metal RTOS with a few fixed task priorities and kilobytes of RAM. The design principle throughout: **one shared way to decide the ordering, two separate ways to enforce it.**

1 Layers and ownership

The work is split across three codebases, and the split is deliberate — each layer answers exactly one question, so the two runtimes (the Linux `play_launch` supervisor and the embedded nano-ros image) can share the parts that must agree and diverge on the parts that cannot.



The pivotal decision (phase-45 §45.10.b): **play_launch is only a parser**. The scheduling **algorithm** is shared as a pure crate; the resolved **plan** is not stored in the model — each runtime re-runs the shared core over the input contracts and then applies **its own** realizer. So Linux and the MCU always agree on the ordering logic, yet each realizes it with the primitives it actually has.

2 The contract — declarative timing intent

The contract is what the mapper reasons over. It is **declarative**: the integrator states the requirement (this path must complete within 20 ms; this one is safety-critical) rather than a scheduling decision. Two ideas are worth separating up front, because they are often conflated:

Criticality $\neq$ priority. **Criticality** is how much the system cares if the work is late (a safety bucket: Low/Medium/High). **Priority** is the mechanical order the scheduler runs things in. The mapper's whole job is to **derive** the second from the first plus timing — criticality is an input, priority is an output. **Where it comes from:** the integrator *declares* criticality per node in the contract manifest — it is authored, never guessed. A chain's criticality is simply the **maximum** over its member nodes (the chain is as critical as its most critical node).

The facts, extracted per launch scope (rlm manifests) and per deployment (system.toml), and projected into the mapper's MapperInput:

Fact	What it means
effective_trigger	How a path is released: Timer{rate_hz} (periodic), Input(srcs) (message-driven), Once, or Spontaneous.
max_latency_ms	The path's end-to-end deadline . For a timer, deadline is implicitly the period ($D = P$).
exec_ms	The path's WCET — worst-case CPU time per run. Distinct from the deadline: how long it takes vs. how long it's allowed.
criticality	Safety importance: Low < Medium < High. A chain inherits the max over its member nodes.
class	Scheduling style: best_effort, real_time, time_triggered, interrupt.
period_us, budget_us	For a sporadic-server budget: how much CPU a callback may spend (budget) before it must wait for the next replenishment (period).
deadline_us, deadline_policy	A runtime deadline monitor and what to do on a miss: ignore warn skip fault.
priority, core, preempt_threshold, stack_bytes	Placement — per-platform, target-specific knobs (posix/freertos/zephyr/threadx/nuttx sub-tables).

Two orthogonal axes, and the mapper keeps them apart: **generic policy** (rates, deadlines, budgets, criticality — portable intent, drives the ranking) versus **platform placement** (priority, core — target-specific, an input to the realizer, never to the ranking).

3 The mapper — from intent to a priority order

The core problem: produce a single priority order that respects criticality, gives tighter-deadline work precedence, and is **feasible**. Done in a platform-agnostic core (chain_aware_rank) whose output carries no OS numbers — just the order — with the OS-specific part left to a realizer.

3.1 Chain, segment, boundary — with a picture

First, the vocabulary, because these three words carry the whole model.

The complete system is a **graph** of nodes wired by topics, and it is a **DAG** — messages fan out (one publisher, many subscribers) and fan in (one node consuming several inputs). A **chain** is

not the whole graph: it is **one causal path** the integrator marks through the DAG as latency-critical, e.g. camera → ... → motor. Below, that path is the top row; /planner also consumes /lidar, so the graph genuinely branches — the chain is just one route through it.

```

/camera -img→ /preproc -feat→ /detector -objs→
(timer 30Hz)
/lidar ----- scan -----
(timer 10Hz)

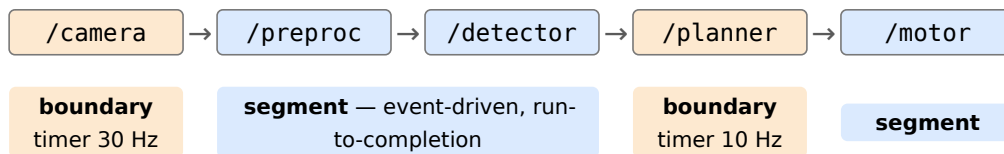
```

/planner -cmd→ /motor
(timer 10Hz)

To schedule a chain the mapper chops it into two kinds of **piece**:

- A **segment** is a **portion of the chain** — a stretch of message-driven nodes that fire back-to-back the instant data arrives, running to completion without waiting.
- A **boundary** is a **timer hop inside the chain** — a node released by a periodic clock, where the pipeline must **wait for the next tick** before continuing. (It is a boundary **between segments**, not between chains.)

So the one chain above decomposes as: /camera is timer-driven (a boundary), then /preproc → /detector run on message arrival (a segment), then /planner waits for its 10 Hz tick (a boundary), then /motor reacts (a segment):



Consecutive boundaries are allowed (two timers back-to-back form one boundary run). A node on several chains belongs to several decompositions — handled below by taking its strongest rank.

3.2 The algorithm, in pseudocode

The whole mapper, idealized (structure, not the real code):

```

def map(chains, loose_nodes, band):          # band = [lo, hi] RT priority window
    order = []                               # ranked list, most-urgent FIRST

    # 1. FEASIBILITY — drop chains whose budget the topology already blows
    feasible = []
    for c in chains:
        sampling = sum(t.period + t.exec for t in c.boundaries) # fixed timer waits
        c.slack = c.budget - sampling                             # schedulable room
        if c.slack > 0: feasible.append(c)
        else: warn("infeasible", c) # no priority can save it

    # 2. ORDER CHAINS — most critical first, then least slack (tightest) first
    feasible.sort(key = lambda c: (-c.criticality, c.slack))

    # 3. RANK INSIDE EACH CHAIN — output side wins; timers rate-monotonic
    for c in feasible:
        for piece in reversed(c.pieces):      # walk sink -> source
            if piece.is_segment: order += reversed(piece.nodes) # drain to sink
            else: order += sort(piece.timers, by="period") # short P
first

```

```

# 4. RANK LEFTOVERS – by criticality, then deadline (period==deadline: RM==DM)
loose = [n for n in loose_nodes if n.has_time_budget]
loose.sort(key = lambda n: (-n.criticality, n.time_budget))
order += loose                                     # no-budget nodes stay non-real-time

# 5. one node, one spot – a node in two chains keeps its BEST (highest) rank
order = dedup_keep_first(order)

# 6. REALIZE – turn rank POSITION into a concrete number, packed into the band
p = band.hi
for node in order:
    node.priority = max(p, band.lo)                # clamp: never below the floor
    p -= 1
# Linux: priority is a SCHED_FIFO level; RTOS: it is the kernel task priority.
# (budget/deadline/timer-windows are enforced by the executor, not here.)

```

Steps 1–5 are the platform-agnostic core (pure ordering); step 6 is the realizer. The rest of this section explains **why** each step is shaped this way.

3.3 Feasibility — why the budget is decomposed

Before ranking, each chain is tested for feasibility, and this is the load-bearing piece of theory. A periodic sampler on the path costs, in the worst case, one full period of waiting plus its own execution — latency that is **architectural**, baked into the topology, and that **no priority assignment can shrink**. So the mapper subtracts that fixed cost from the declared budget and asks whether any slack remains for the parts scheduling can actually shape:

```

sampling_cost = Σ over timer boundaries (periodi + execi)
controllable = max_latency_ms - sampling_cost (feasible iff > 0)

```

If $\text{controllable} \leq 0$ the budget is impossible on this topology regardless of priorities; the chain is flagged (`ChainInfeasible`) and dropped from priority shaping — its nodes fall back to the ordinary path. This is a **latency-budget** argument, not a CPU-utilization one: the contract constrains end-to-end time, so the right question is “does the unavoidable sampling cost fit inside the budget?”, not the classic Liu-Layland “does the CPU saturate?”.

3.4 Ranking — the ordering logic

The core ranks in the order most-urgent-first, purely as positions (no numbers yet):

- **Between chains:** highest criticality first; ties broken by **tightest slack** first (smallest `controllable`), then name. The chain with the least room to spare gets the strongest claim on the CPU.
- **Within a chain:** walk **sink** → **source** and give the downstream (output) side the higher rank — **drain toward the sink**. Intuition: once data is in flight, let it reach the actuator without being preempted by fresh work entering upstream, which minimizes end-to-end latency. Within a run of adjacent timer boundaries, order them **rate-monotonic** (shorter period → higher).
- **Everything not in a chain:** each remaining (node, path) that has a usable time budget — a timer’s period, or an input path’s declared `max_latency_ms` — is ranked by criticality bucket, then by that single millisecond budget, then name. Paths with no budget (`Once`, `Spontaneous`, `Unclassified`, or an input path with no declared deadline) are left unranked and fall to a non-real-time default.

- **Per node:** a node appearing on several paths takes the **maximum** rank it earns anywhere (a node shared by two chains inherits the stronger claim).

Why one sort covers timers and event paths (RM $\equiv$ DM). Under the implicit-deadline assumption $\text{deadline} = \text{period}$, **rate-monotonic** (shorter period $\rightarrow$ higher priority) is identical to **deadline-monotonic** (shorter deadline $\rightarrow$ higher). That equivalence lets a timer's period and an event path's declared deadline be compared in the same millisecond unit: a 50 ms-deadline input correctly outranks a 100 ms timer, and a 10 ms timer outranks that input. One sort, both trigger kinds, no fudge.

3.5 The Linux realizer — packing the order into a priority band

The order is abstract; Linux needs concrete SCHED_FIFO integers. But real-time priorities are a scarce, shared resource, which is what a **band** is about:

What is a priority band? Linux real-time priorities run 1–99, and **every** RT program on the box competes for them — audio, network, watchdogs. An integrator therefore reserves a contiguous **sub-range**, e.g. [5, 45], that this system's nodes may occupy, so nano-ros/play_launch coexist with the rest instead of monopolizing the RT range. The realizer must fit its **entire** ranked list into that window. If the order has more distinct levels than the band has integer slots, it **compresses** — merging adjacent ranks into ties — to make it fit.

realize_posix dense-ranks the order into descending priorities starting at band.max. When the band is too narrow it collapses adjacent groups in a strictly legal sequence — first within a **segment** (fine_group), then within the same **chain** (coarse_group), **never** across a criticality bucket or the chain / non-chain divide — and if it still overflows, clamps the lowest classes onto band.min, emitting BandTooNarrow. The invariant it protects: compression may introduce **ties**, **never inversions** — two nodes may end up equal, but a lower-ranked node never overtakes a higher one. Output is a SCHED_FIFO tier per node; unranked nodes get SCHED_OTHER. It stays fixed-priority — it does **not** emit SCHED_DEADLINE (EDF).

Worked example (from the crate's pinned test). Chain sensing_to_actuation, budget 150 ms, band 5..45. Sink-first dense-ranking gives the output side the top priorities; the sampling timer lands just below the chain but stays real-time; a one-shot loader is unranked $\rightarrow$ SCHED_OTHER:

/gate	/follower	/ekf	/planning	/detector	/concat	/preproc	sim_timer	loader
45	44	43	42	41	40	39	< 39	OTHER

4 RTOS realization (nano-ros)

nano-ros consumes the **same** ranked facts but cannot assume Linux's scheduler. Its realizer **splits** the contract across two mechanisms — one in the kernel, one in the executor — because small RTOS kernels give you preemptive fixed-priority tasks but **not** the richer reservation and deadline machinery.

Why part of it is cooperative. A kernel like FreeRTOS or ThreadX will preempt a lower-priority task for a higher one — that part is native. But it has no notion of “run this callback only until it has spent 2 ms of CPU this period” (**sporadic server**), or “release this work only inside a fixed time window” (**time-triggered** / logical-execution-time), or “skip this callback because it missed

its deadline”. nano-ros implements those **in software**, inside its executor’s `spin_once` loop, checking budget and deadline between callbacks and voluntarily skipping — hence **cooperative**. It backfills what the kernel lacks.

Build-time wiring. Codegen bakes a `&[TierSpec]` slice — `class`, `period_us`, `budget_us`, `deadline_us`, `deadline_policy`, `priority`, `preempt_threshold`, `core` — into the image. `nros::main!` calls `Board::run_tiers`, which spawns one task per tier and splits each `TierSpec`:

(a) priority → kernel. The tier’s priority is handed to the native task-create. On **ThreadX**, **FreeRTOS**, and **Zephyr** this is a real kernel task and the priority is the actual scheduling priority (ThreadX also carries a **preemption threshold**). The value is the **raw** per-kernel number — the author writes it directly.

(b) budget/deadline/timing → executor. `SchedContext::from_tier_policy` classifies once (shared by the Rust runtime and the C++ FFI): `real_time+budget+period` → **sporadic**; `best_effort` → best-effort; `time_triggered+period` → a time-triggered **frame**; `deadline_us` → a deadline monitor with action `Ignore/Warn/Skip/Fault`.

Runtime enforcement lives in `Executor::spin_once` (portable Rust, identical on every target): a sporadic-budget gate stops dispatching a context’s callbacks once its budget is spent until the next period; a time-triggered gate releases callbacks only inside their window; a per-callback overrun check fires the `DeadlineAction` (`Fault` → board fault hook, else `panic!`).

4.1 The asymmetry

Which half reaches the kernel depends on the target, and it is **not** a clean Linux-vs-RTOS line:

Target	Priority (preemption)	Budget / deadline / TT
ThreadX, FreeRTOS, Zephyr	native kernel priority (real preemption)	cooperative in <code>spin_once</code>
NuttX, native/POSIX (incl. Linux)	advisory only — tiers are ordinary scoped OS threads; priority is not pushed to the kernel	cooperative in <code>spin_once</code>

So on ThreadX/FreeRTOS/Zephyr the priority order is genuinely enforced by the kernel **and** the budget/deadline shape is enforced cooperatively. On NuttX and native/POSIX the tiers run as plain threads and only the cooperative layer is active — nano-ros-on-Linux is for functional testing, not RT guarantees; hard priority on Linux is `play_launch`’s job via the band realizer above. Two loose ends on the record: the RFC-0016 priority-**normalization** helpers (mapping a 0–31 abstract scale onto each kernel’s native range and direction) exist but are currently **unwired** — the integrator writes the kernel-native value directly; and a `SCHED_FIFO/SCHED_DEADLINE` worker is wired into `spin_once` but never **armed** outside tests, so native Linux issues no RT syscalls today.

4.2 Design trajectory (RFC-0052, Draft)

The implemented RTOS realizer keys off the tier `class`. The design generalizes it to a **six-dimension** requirement per causal segment — `activation`, `urgency`, `deadline`, `budget`, `non_preempt_scope`, `placement` — each resolved at build time against a board’s declared capabilities to `Native` | `Backfill` | `Degrade(recorded)`, **fail-loud** (a knob the board cannot honor is either backfilled by the executor or recorded as a degradation, never silently dropped),

over a three-layer stack: an L1 host-side realizer (6-dim × board CAPS), a thin L2 PlatformSched board trait, and the portable L3 executor. The unit becomes the **causal segment** — one executor, run-to-completion — with the current fixed-priority tier table as the fallback. The shared `chain_aware_rank` core feeds both.

Grounding (per RFC-0052). Fixed-priority chain→priority ranking follows **PiCAS** (RTAS '21); segment = one run-to-completion executor thread mirrors micro-ROS's **rlc** executor (LET + static order); reservation-based chains + callback-group concurrency follow **Casini et al.** (ECRTS '19), which also underlies the crate's `contract-theory.md`. **Sources.** `chain_aware_mapper.rs`, `chain.rs`, `mapper.rs`, `resolve.rs`, `platform.rs`, `docs/scheduling.md` (ros-launch-manifest); `sched_context.rs`, `spin.rs`, `node_runtime.rs`, `board/tier.rs`, `nros-board-{threadx, freertos, zephyr, nuttx, posix}`, `nros-platform-*` (nano-ros); RFC-0047, RFC-0050, RFC-0052. **Implemented:** the `chain_aware_rank/realize_posix` split, `TierSpec` → `SchedContext` + cooperative `spin_once` enforcement, native task priority on ThreadX/FreeRTOS/Zephyr. **Designed (RFC-0052 Draft):** the six-dim RTOS realizer / L1-L3 stack; priority normalization and native-Linux RT syscalls are unwired.